

ADR-COM-15-HUB MI Data Views

Status	In Progress
Owner	@Namdi Onwuachu
Date	Jun 19, 2025

Deploying Amazon QuickSight Dashboards Across Multiple AWS Accounts with Asset Bundles, GitHub for Configuration Management, Athena, and Multi-Account User Access [↗](#)

Context [↗](#)

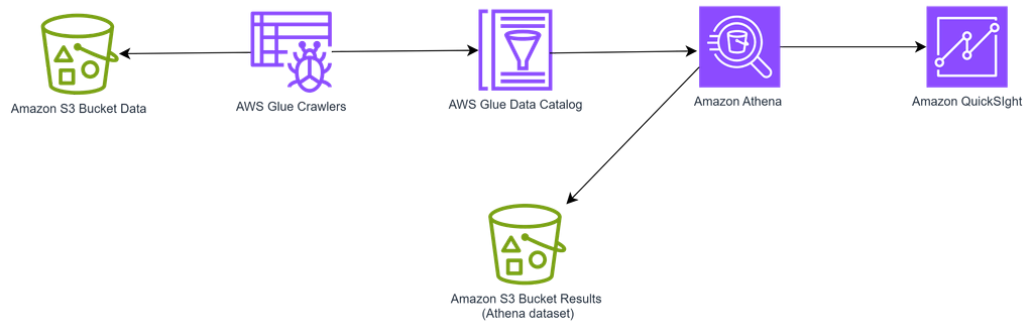
The Kaizen organisation intends to deploy Amazon QuickSight dashboards across 200 AWS accounts to deliver standardized dashboards layout and themes across all client AWS accounts and including its subsidiaries. However, each client will only visualise their data. The deployment must address scalability, data sovereignty, user access, cost-efficiency, and security. Additionally, embedding dashboards into client applications is required for seamless user experience.

Challenges include:

1. **Data Sovereignty:** Ensuring datasets reside in client-accounts.
2. **Administrative Overhead:** QuickSight lacks CloudFormation support for QuickSight Installation and user provisioning.
3. **Embedding Dashboards:** Enabling intuitive access.
4. **Cost Efficiency:** Managing costs for TBC (population of users etc.) 1,000 Readers (5 per account) and 3 Authors. (The headcount of users Authors, readers need to be confirmed).
5. **Scalability:** Deploying dashboards consistently across multiple accounts.

Architecture Schematic [↗](#)

Solution Diagram



Schematic 1

Decision [🔗](#)

Dashboard Template Management with Asset Bundles [🔗](#)

1. Centralized Template Creation

- Develop baseline Amazon QuickSight dashboards in a source AWS account.
- Use `StartAssetBundleExportJob` to export dashboards, analysis, and themes as asset bundles.
- Store the exported JSON in an S3 bucket with versioning enabled for historical tracking.
- Push JSON files to Github repository for version control and change tracking.

2. Set Up Target AWS Accounts

- Enable Amazon QuickSight and configure Admin/User roles as needed.
- Create an S3 Bucket in each target account to store imported asset bundles.
- Ensure IAM roles grant necessary permissions (e.g. read access to the respective S3 Bucket and QuickSight API permissions).

3. Dataset Preparation in Client AWS Account

- Ensure **client datasets** are available in **Amazon S3** and organized by prefixes or partitions for efficient querying.
- Use AWS Glue crawlers to scan and catalog datasets into the AWS Glue Data Catalog.
- Configure Amazon Athena to query the cataloged datasets dynamically.
- Apply IAM policies to restrict dataset access to only relevant QuickSight users.
- Enable SPICE (Super-fast, Parallel, In-memory Calculation Engine) where necessary for faster in-memory data retrieval.

4. Deploy Asset Bundles to Target Accounts

- Use `StartAssetBundleImportJob` to import dashboards and analysis.
- Configure dataset ARNs in the target account QuickSight setup.
- Deploy Cloudformation template with custom resource to create dashboard based on analysis and client's unique dataset.

d. Validate dashboards and ensure dataset mappings are correct.

5. Embedding Dashboards

a. API Integration:

b. Use the import boto3 GenerateEmbedUrlForRegisteredUser API to generate embed URLs for registered users.

c. Policy Enforcement:

i. Update CORS (Cross-Origin Resource Sharing) and CSP (Content Security Policy) to allow embedding without compromising security.

6. User Management

a. Author and Reader Segmentation:

i. Designated authors centrally manage dashboard creation and updates in the source account.
ii. Each of the 200 target accounts will have Readers with secure access to dashboards.

7. Security

a. Data Protection:

i. Implement strict IAM roles.
ii. Apply encryption mechanisms across the stack e.g., (S3 bucket, AWS Glue, Athena) for secure operations.
iii. Ensure client datasets remain in their respective AWS accounts to maintain data sovereignty.
iv. Apply decryption mechanisms at QuickSight using the IAM role permissions.

8. Automation

a. Deployment Automation

i. Automate user provisioning and subscription management through scripts.
ii. Automate the export/importing of the Amazon QuickSight asset bundles from source account to client AWS accounts.
iii. Use parameterized CloudFormation templates for account-specific deployments.

9. Performance

a. Datasource

i. We would leverage Athena SPICE (Super-fast, Parallel, In-memory Calculation Engine) compared to direct import querying generally offers several performance advantages:
1. in-memory processing speed
2. cached results

b. Important performance considerations:

i. Initial import latency - First data load depends on size/complexity
ii. Memory constraints - Limited by QuickSight edition and SPICE capacity
iii. Query overhead - Complex joins and calculations can impact performance

10. Dataset Refresh (waiting on confirmation)

Data updates will be handled autonomously at a defined cadence using AWS-native services. The refresh pipeline will include the following components:

a. **Amazon S3:** Event notifications will detect the arrival of new data (e.g., the creation of new test_run partition prefixes).

- b. **AWS Glue:** Crawlers will update the data catalog and optionally Glue jobs will transform the new partitions.
- c. **Amazon Athena:** Provides consistent SQL query access to the transformed data via the Glue Data Catalog.
- d. **Amazon QuickSight:** SPICE datasets will be refreshed asynchronously using Athena import mode, ensuring the visual layer stays current with minimal latency.

11. Governance, Observability, and Monitoring

- a. **Versioning:** Although asset bundles will be downloaded stored in S3 buckets and Github for configuration management purposes.
- b. **Monitoring:** Track dashboard usage (views, updates) with QuickSight Usage Metrics or CloudWatch. Use AWS Config for compliance with data sovereignty and log API calls via CloudTrail and S3 access.
- c. **Auditing:** Conduct regular reviews of IAM policies, dataset mappings, SPICE usage, and user access to ensure security, compliance, and cost optimization.
- d. **Automation:** Enable alerts for anomalies like SPICE overuse, unauthorized access, or compliance violations using CloudWatch and SNS notifications.

12. SPICE Capacity Limits:

- a. Amazon QuickSight can handle up to 1 billion rows or 1 TB of data per dataset in the Enterprise Edition.

13. Cost Management

a. Named User Licensing:

- i. Estimated annual cost: \$232,800 for 3 Authors and 5 Readers per member account across 200 accounts. (The headcount of users Authors, readers need to be confirmed).
- ii. Cost optimization strategies include monitoring active user licenses and reviewing access requirements periodically.

b. SPICE Capacity pricing:

- i. Authors receive 10 GB of SPICE capacity for free per month in Amazon QuickSight Enterprise Edition. (The headcount of users Authors, readers need to be confirmed). Kaizen may loose the SPICE free capacity based on current discussions of using Authors for pretty limited durations.
- ii. Client accounts (Readers) do not have any free SPICE capacity. Readers are charged at \$3 per month per reader plus amounts specified in the table below.

- c. **SPICE Storage Costs:** Costs are calculated (per account) based on \$0.38 per GB per month.

SPICE GB (Storage)	Records Volumes (millions)	1 Month (\$)	1 Year (\$)
5	1	1.9	22.8
10	5	3.8	45.6
20	10	7.6	91.2
50	25	19	228
100	50	38	456

Notes:

1. Costs are based on estimated Kaizen author and reader numbers.
 2. Cost estimation for QuickSight excludes:
 - a. Enterprise Edition with Capacity pricing
 - b. Amazon Q's Generative BI features
 - c. QuickSight Enterprise Edition with Capacity Pricing (Anonymous user access)
 3. Enforcement of Row-Level Security (RLS) to ensure secure, user-specific access to data is not applicable.
-

Consequences

Positive Outcomes

1. **Security:** Eliminates cross-account access complexities, localizes IAM roles, and enforces strict data access controls.
2. **Governance:** Strengthens governance through GitHub version control mechanisms for asset bundles.
3. **Scalability:** Enables consistent deployment across accounts with reusable templates, ensuring a streamlined expansion process.
4. **Reduced Risk:** Testing in controlled environments (dev, test) minimizes deployment failures, aligns with Kaizen's progressive deployment approach, and leverages segregation principles.
5. **Efficiency:** Parameterized templates and automation significantly reduce manual effort, improving operational efficiency.
6. **User Access:** Embedded dashboards simplify client access by removing login barriers, enhancing user experience.

Negative Trade-offs

1. **High Administrative Overhead:** Manual provisioning is required due to the lack of native Infrastructure as Code (IaC) support for QuickSight installation and user setup. Additionally, asset bundles must be stored in a configuration management system and manually transferred across target accounts. There is currently no API to assign IAM roles programmatically for QuickSight setup, so IAM roles must be manually bound per installation.
 2. **Ongoing Development Effort:** Maintaining automation scripts and embedding APIs demands continuous updates and active monitoring.
 3. **Limited Customization:** Using embedded dashboards within iframes restricts frontend design flexibility, limiting the ability to tailor user experience and interface elements.
 4. **AWS Service Quotas (Ingestion Serialization Constraint):** QuickSight enforces a one-at-a-time ingestion limit per dataset. Even across different accounts, automated deployments may encounter race conditions due to ingestion timing conflicts during asset bundle imports, requiring custom logic to await ingestion completion before proceeding.
-

Alternatives Considered

1. **Cross-Account IAM Role for QuickSight:**

- **Pros:** Cost-effective (admin licenses only in source account), secure access control, auditable, supports automation.
 - **Cons:** Complex initial setup, requires IAM expertise, needs consistent role maintenance across accounts.
2. **Centralized QuickSight Account:**
- **Pros:** Simplifies user management.
 - **Cons:** Violates data sovereignty and limits scalability.
3. **Manual Setup:**
- **Pros:** No development effort.
 - **Cons:** Time-intensive, prone to errors, and lacks scalability.
4. **Custom Automation Scripts:**
- **Pros:** Balances effort and flexibility.
 - **Cons:** Requires development and maintenance.
-

Related Decisions [↗](#)

- To be updated

Reference Sources [↗](#)

1. <https://aws.amazon.com/solutions/guidance/multi-account-environments-on-amazon-quicksight/>
2. <https://aws.amazon.com/blogs/business-intelligence/automate-and-accelerate-your-amazon-quicksight-asset-deployments-using-the-new-apis/>
3. <https://aws.amazon.com/blogs/big-data/migrate-amazon-quicksight-across-aws-accounts/>

Votes [↗](#)

Names	Vote	Comments